

ReqSoftware Engineering Lab: From Personas & Requirements to Use Cases & UML

Course: CS3151 Software Engineering

Lab Duration: 2–3 hours (with a 1-week post-lab)

Context: You already have a Business Model Canvas, initial requirements, and customer personas for your project. In this lab you will transform these into **use cases** and a consistent set of **UML artifacts**.

Team: New Group 8 EquRental — Alaa Alqurashi, Faisal Alburti, Hashim Alsayed

Project: Equipment Rental Management System (EquRental) — tracks equipment availability, rentals, returns, overdue items, maintenance status, customer records and payments for small/medium rental businesses (bilingual Arabic/English). Based on a real family equipment rental business.

Course / inputs: Effat University, CS3151 Software Engineering — Lab 4: Use Cases & UML. Builds on our Lab 1 market study, Lab 2 personas/user stories and Lab 3 SMART requirements (US-01...US-08, ER-01...ER-19).

Learning Objectives

By the end of this lab, you will be able to:

1. Identify **actors** and **user goals** from personas and requirements.
 2. Define and prioritize a **use case list**; write both **brief** and **fully dressed** use cases.
 3. Draw a **Use Case Diagram** with correct system boundary, includes/extends, and generalization.
 4. Elaborate behavior with **Activity** and **Sequence Diagrams** for key scenarios.
 5. Extract a **Domain Model (Class Diagram)** from the problem domain (not the database schema).
 6. Check **consistency** across artifacts and run a structured **peer review**.
-

Deliverables (What to Submit)

Upload a single PDF named TeamName_UML_Lab.pdf containing: -

D1. Use Case List (prioritized with MoSCoW or similar) and **Actor definitions**

D2. Two fully dressed Use Case specifications (one primary, one alternate/edge)

D3. Use Case Diagram

D4. Activity Diagram for the most complex use case

D5. Sequence Diagram for the main success scenario

D6. Domain Model (Class Diagram)

D7. Consistency checklist + Peer-review sheet

(Optionally, include PlantUML source in an appendix.)

Tooling (Choose any)

- **Draw.io/diagrams.net, StarUML, PlantUML, Lucidchart, Visual Paradigm**
 - If using PlantUML, keep code blocks in your appendix for reproducibility.
-

Lab Steps & Activities

Step 0 — Warm-up (10 min)

Quickly re-read your **personas** and **top 10 user stories/requirements**. Highlight verbs (actions) and nouns (entities). Keep these highlights visible.

Our verbs (→ use cases): view/search, rent, record, return, inspect, mark (under maintenance), pay, issue (receipt), add/update, follow up (overdue).

Our nouns (→ domain classes): equipment, status, rental, due date, return condition, customer, payment, receipt, balance, rental request, staff.

Sources re-read: Lab 3 user stories US-01 to US-08 (requirements ER-01 to ER-19) and the six Lab 2 persona stories for Ahmed Al-Harbi, Khalid Hassan and Omar Saleh.

Step 1 — Identify Actors & Goals (25 min)

1. List all **external actors** (human roles, systems, devices, timers).
2. For each persona, map to one or more actors.
3. Derive **user goals** (what success looks like for each actor).

Template: Actor Card

Actor	Description	Triggers/Interests	Challenges	Notes
Rental Clerk	Shop employee at the rental counter; primary actor for daily operations.	Customer at the counter wants to rent or return an item; wants fast availability checks and accurate rental, return and payment records.	Manual checks, wrong rental details, spotting overdue items while busy.	Persona: Khalid Hassan (27, rental shop employee). Primary actor of UC-1, 2, 3, 5, 6, 7, 8.
Business Owner	Owner/manager of the business. Specialization of Rental Clerk: can do everything a clerk does.	Wants to know where every item is, recover overdue equipment, keep damaged items out of rental, follow payments.	Paper records, forgotten return dates, late payments, lost equipment.	Persona: Ahmed Al-Harbi (42). Inherits all clerk use cases; own use cases UC-9, UC-12.
Customer	Person or small business renting equipment; uses the public catalog on a phone.	Needs equipment for a project; wants to see availability and price and reserve without calling.	Unclear availability and prices, slow rental process.	Persona: Omar Saleh (31). Primary actor of UC-10, UC-11; offstage stakeholder in UC-1 and UC-2 (served by the clerk).
System Clock (timer) — future	Time-based trigger outside the system.	Would start daily overdue reminders.	—	Not used in this release: overdue is calculated when the list is opened (ER-08). Needed only for UC-13 (Won't).
Payment Gateway (external system) — future	Online card payment service.	Would charge cards online.	—	Out of scope: Lab 3 records payments at the counter only (no online card processing). Needed only for UC-14 (Won't).

Persona → actor mapping and user goals (what success looks like)

Persona (Lab 2)	Actor	User goals / success
Ahmed Al-Harbi — Business Owner	Business Owner (inherits Rental Clerk)	Sees all overdue rentals with customer phone in one list; damaged items are Under maintenance and cannot be rented; payments and balances are recorded.
Khalid Hassan — Rental Shop Employee	Rental Clerk	Creates a rental in one flow and gets a rental ID and due date; closes returns with the right equipment status; records payments and prints receipts.
Omar Saleh — Customer	Customer	Finds available equipment and the daily rate on his phone (Arabic or English); sends a rental request for his dates; gets a clear due date and receipt from the shop.

Step 2 — Draft the Use Case List (20 min)

Create a candidate list of **use cases** (goal-level). Avoid UI details; think outcomes.

Checklist: -

- Named as **Verb + Noun** (e.g., *Place Order*, *Schedule Pickup*)

- Scope is the **system**, not a single page/button
- Prioritize with **MoSCoW** (Must, Should, Could, Won't for now)

Template: Use Case List

ID	Use Case Name	Primary Actor	Goal/Outcome	Priority (M/S/C/W)
UC-1	Rent Equipment	Rental Clerk	Rental recorded for a customer; item set to Rented; rental ID and due date shown	M
UC-2	Return Equipment	Rental Clerk	Rental closed; item Available or Under maintenance depending on condition	M
UC-3	Search Available Equipment	Rental Clerk	Available items listed by ID, name or type	M
UC-4	Verify Equipment Availability («include»)	— (included by UC-1, UC-11)	Item confirmed still Available before a rental or request is saved	M
UC-5	Record Payment	Rental Clerk	Payment saved, balance updated, receipt printed	M
UC-6	Manage Customer Records	Rental Clerk	Customer added or contact details updated (unique customer ID)	M
UC-7	Manage Equipment Records	Rental Clerk	Equipment added or updated (unique equipment ID)	M
UC-8	View Overdue Rentals	Rental Clerk, Business Owner	Unreturned past-due rentals listed with customer phone for follow-up	M
UC-9	Update Maintenance Status	Business Owner	Item moved between Available and Under maintenance	M
UC-10	Browse Equipment Catalog	Customer	Available equipment and daily rates shown in Arabic or English	S
UC-11	Submit Rental Request	Customer	Request for an item and dates sent to the shop for staff confirmation	C
UC-12	View Business Reports	Business Owner	Rental and payment summary for a chosen period	C
UC-13	Send Overdue Reminders	System Clock	Automatic SMS/e-mail reminder to customers with overdue items	W
UC-14	Pay Online by Card	Customer (Payment Gateway supports)	Customer pays online by card	W

Priority rationale: M = covered by our Lab 3 SMART requirements (core counter workflow). S/C = Lab 2 persona features not yet written as SMART requirements. W = excluded by the Lab 3 scope note (no online card processing, no automatic late fees/notifications) — kept for a later release and not drawn in the diagram.

Traceability matrix.

UC	Use case	User story (Lab 2 / Lab 3)	Requirements (Lab 3)	Persona
UC-1	Rent Equipment	US-02; Khalid story 2	ER-03, ER-04, ER-05	Khalid
UC-2	Return Equipment	US-03; Khalid story 2	ER-06, ER-07	Khalid

UC-3	Search Available Equipment	US-01; Khalid story 1	ER-01, ER-02	Khalid
UC-4	Verify Equipment Availability	US-02	ER-04	Khalid, Omar
UC-5	Record Payment	US-07	ER-15, ER-16, ER-17	Khalid, Ahmed
UC-6	Manage Customer Records	US-08	ER-18, ER-19	Khalid
UC-7	Manage Equipment Records	US-05	ER-10, ER-11	Khalid, Ahmed
UC-8	View Overdue Rentals	US-04; Ahmed story 1	ER-08, ER-09	Ahmed, Khalid
UC-9	Update Maintenance Status	US-06	ER-12, ER-13, ER-14	Ahmed
UC-10	Browse Equipment Catalog	Omar story 1	— (to be specified)	Omar
UC-11	Submit Rental Request	Omar story 2	— (to be specified)	Omar
UC-12	View Business Reports	Ahmed story 2	— (to be specified)	Ahmed
UC-13	Send Overdue Reminders	Ahmed usage scenario (contact overdue customer)	Out of scope (Lab 3)	Ahmed
UC-14	Pay Online by Card	Omar motivation (no shop visit)	Out of scope (Lab 3 payment note)	Omar

Step 3 — Write Use Case Specifications (35 min)

Select **two** use cases: one core (happy path) and one complex/edge.

Our selection: UC-1 Rent Equipment (core, happy path — the main counter task) and UC-2 Return Equipment (complex/edge — overdue returns, damaged items, outstanding balance).

A. Brief (for all use cases)

Use Case	Primary Actor	Pre-conditions	Post-conditions	Main Success Scenario (1–2 lines)
UC-1 Rent Equipment	Rental Clerk	Clerk logged in; customer and equipment records exist	Rental saved; item Rented	Clerk selects customer and item, enters dates and charge, confirms; system verifies availability, saves the rental and shows rental ID and due date.
UC-2 Return Equipment	Rental Clerk	Active rental exists	Rental closed; item Available or Under maintenance	Clerk finds the rental, enters return time and condition; system closes the rental and updates the item status.
UC-3 Search Available Equipment	Rental Clerk	Clerk logged in	Matching Available items shown, or “No available equipment found”	Clerk searches by ID, name or type; system lists Available items with ID, name and type.
UC-4 Verify Equipment Availability	— (included)	Equipment ID given by UC-1 or UC-11	Item confirmed Available, or the base use case is stopped	System checks the item status is Available and it has no active rental.
UC-5 Record Payment	Rental Clerk	Rental exists with balance > 0	Payment saved; balance updated; receipt shown	Clerk enters amount and method; system checks $0 < \text{amount} \leq \text{balance}$, saves the payment and prints a receipt.
UC-6 Manage Customer Records	Rental Clerk	Clerk logged in	Customer saved with unique ID; rental links kept	Clerk enters or edits name and phone; system validates and saves.

UC-7 Manage Equipment Records	Rental Clerk	Clerk logged in	Equipment saved with unique ID	Clerk enters ID, name, type and initial status; system rejects duplicates and saves.
UC-8 View Overdue Rentals	Rental Clerk, Business Owner	User logged in	Overdue list shown (no data changed)	User opens the list; system shows each unreturned past-due rental with customer name, phone and due date.
UC-9 Update Maintenance Status	Business Owner	Item is not on an active rental	New status saved (Available ↔ Under maintenance)	Owner selects the item and new status; system saves it and updates availability.
UC-10 Browse Equipment Catalog	Customer	Catalog is online	None (read only)	Customer filters by type in Arabic or English; system shows available items and daily rates.
UC-11 Submit Rental Request	Customer	Customer gives name and phone	Request saved as Pending for staff	Customer picks an item and dates; system verifies availability and saves the request.
UC-12 View Business Reports	Business Owner	Owner logged in	None (read only)	Owner selects a period; system shows rentals, payments and outstanding balances.

B. Fully Dressed (for 2 selected cases)

Each activity in the UC

UC-1 Rent Equipment — core use case (primary, happy path)

Field	Content
Use Case	UC-1 Rent Equipment
Activity Name	Record a counter rental (create the rental and mark the equipment Rented)
Primary Actor	Rental Clerk (persona Khalid Hassan)
Stakeholders & Interests	Rental Clerk – quick, error-free rental at the counter. Customer (Omar, offstage) – gets the right item, a clear due date and the agreed charge. Business Owner (Ahmed) – every item that leaves the shop is recorded and never rented twice.
Preconditions (Ip)	Clerk is logged in; customer and equipment records exist (ER-10, ER-18). Inputs: customer ID, equipment ID, start date/time, due date/time, agreed rental charge.
PostConditions (Op)	One unique rental record is saved, linked to the customer and the equipment, and the equipment status is Rented — both saved as one operation (ER-05). Output: rental ID and due date displayed.
Main Scenario (Flow of Control)	1) Clerk searches for the customer by name or phone. 2) System shows matching customers; clerk selects one. 3) Clerk enters the equipment ID (found with UC-3 if needed). 4) Clerk enters start date/time, due date/time and agreed rental charge. 5) Clerk confirms the rental. 6) System validates the details (ER-03). 7) System verifies the equipment is still Available — «include» UC-4 (ER-04). 8) System creates the rental and sets the equipment to Rented (ER-05). 9) System displays the rental ID and due date.

Extensions (Alternatives)	2a) Customer not found → 2a1) Clerk adds the customer with UC-6 Manage Customer Records (ER-18); 2a2) resume at step 2. 6a) Missing or invalid field (due not later than start, negative charge) → 6a1) System highlights the field and blocks confirmation; 6a2) clerk corrects and resumes at step 5. 7a) Equipment is Rented or Under maintenance (or another clerk just rented it) → 7a1) System shows “Equipment not available” and saves nothing; 7a2) clerk picks another item with UC-3 and resumes at step 3, or cancels. 8a) Save fails → 8a1) System rolls back: neither the rental nor the status change is saved; error shown. 9a) Customer pays at pickup → 9a1) «extend» UC-5 Record Payment; receipt printed.
Special Requirements	Bilingual Arabic/English screens (right-to-left for Arabic); confirmation shown within 3 seconds; works in desktop and phone browsers; only logged-in staff can create rentals.
Constraints	One active rental per item; item must be Available; due date/time later than start; agreed charge ≥ 0; one item per rental in this version; no online card payment and no automatic late fees (Lab 3 scope).
Open Issues	Should one rental hold several items (e.g., generator + cables)? Is a deposit needed? How does an accepted UC-11 request become a rental?

UC-2 Return Equipment — complex / edge use case

Field	Content
Use Case	UC-2 Return Equipment
Activity Name	Process an equipment return (close the rental, record condition, update equipment status)
Primary Actor	Rental Clerk (persona Khalid Hassan)
Stakeholders & Interests	Rental Clerk – closes the return quickly and correctly. Business Owner (Ahmed) – damaged items are not rented again, overdue items are recovered, balances are collected. Customer (Omar, offstage) – rental closed correctly and he knows what he still owes.
Preconditions (Ip)	Clerk is logged in; the rental is active (not yet returned). Inputs: rental ID or equipment ID, actual return date/time, condition (Good / Needs maintenance).
PostConditions (Op)	Rental closed with return date/time and condition; equipment Available (Good) or Under maintenance (Needs maintenance), saved as one operation (ER-07); the rental no longer appears on the overdue list. Output: return confirmation with equipment status and remaining balance.
Main Scenario (Flow of Control)	1) Clerk looks up the active rental by rental ID or equipment ID. 2) System displays the rental details (customer, equipment, due date, balance). 3) Clerk enters the actual return date/time. 4) Clerk inspects the equipment and records condition Good. 5) Clerk confirms the return. 6) System validates the return data (ER-06). 7) System closes the rental and sets the equipment to Available (ER-07). 8) System displays the return confirmation and remaining balance (zero).

Extensions (Alternatives)	1a) No active rental found, or rental already returned → System shows “No active rental found”; use case ends. 2a) Due date/time already passed → System flags the rental as overdue and shows days late; continue at step 3 (no automatic late fee in this version). 4a) Equipment needs maintenance → 4a1) Clerk records “Needs maintenance”; 4a2) at step 7 the system sets the status to Under maintenance, so the item is excluded from availability. 6a) Missing data or return time before the rental start → System rejects and highlights the fields; resume at step 3. 8a) Outstanding balance > 0 → «extend» UC-5 Record Payment; receipt shows the remaining balance.
Special Requirements	Bilingual Arabic/English; return confirmed within 3 seconds; condition chosen from a fixed list (Good / Needs maintenance).
Constraints	Only active rentals can be returned; return time not before rental start; while an item is rented its status can change only through this return process (ER-14).
Open Issues	Should late returns add a late fee in a later version? Should a damage note or photo be stored with the condition?

Step 4 — Draw the Use Case Diagram (20 min)

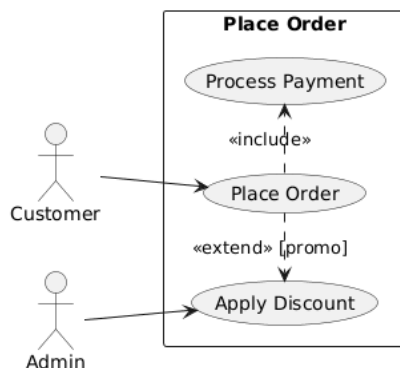
Include **system boundary**, actors, associations, and use relationships (**«include»**, **«extend»**, generalization when justified).

PlantUML Example

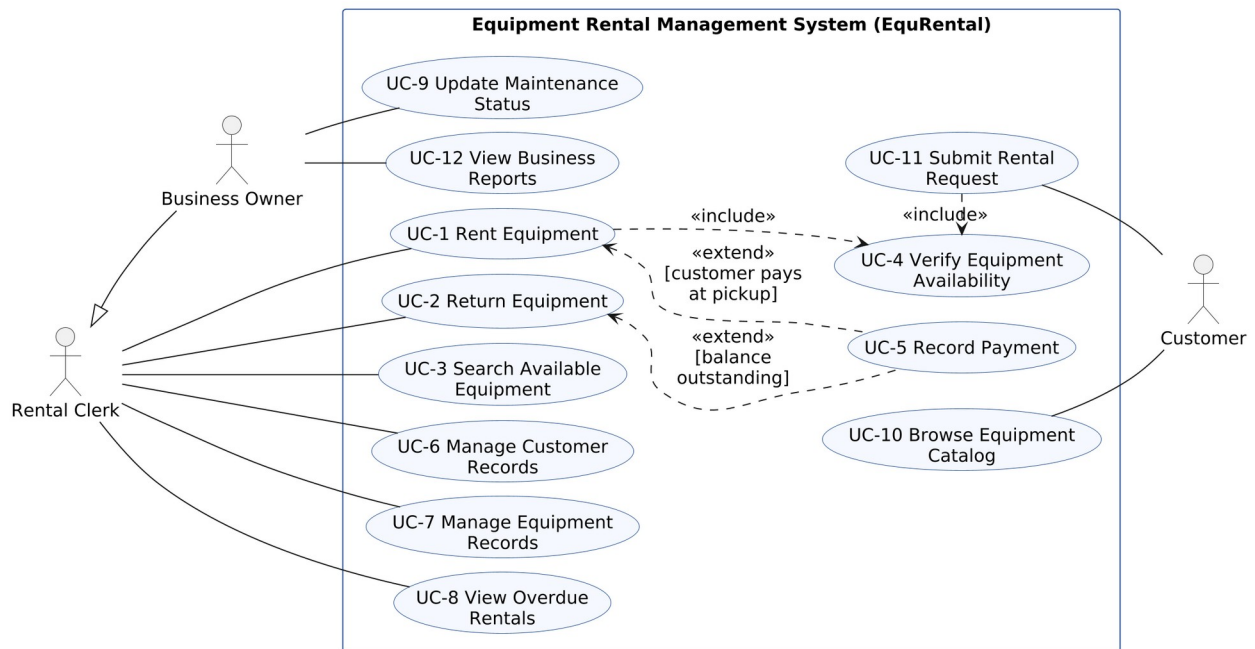
```

@startuml
left to right direction
actor Customer
actor Admin
rectangle "UC1- Place Order" {
    usecase "Place Order" as UC1
    usecase "Apply Discount" as UC2
    usecase "Process Payment" as UC3
}
Customer --> UC1
UC1 .-> UC3 : <<include>>
UC1 .-> UC2 : <<extend>> [promo]
Admin --> UC2
@enduml

```



D3 — EquRental Use Case Diagram

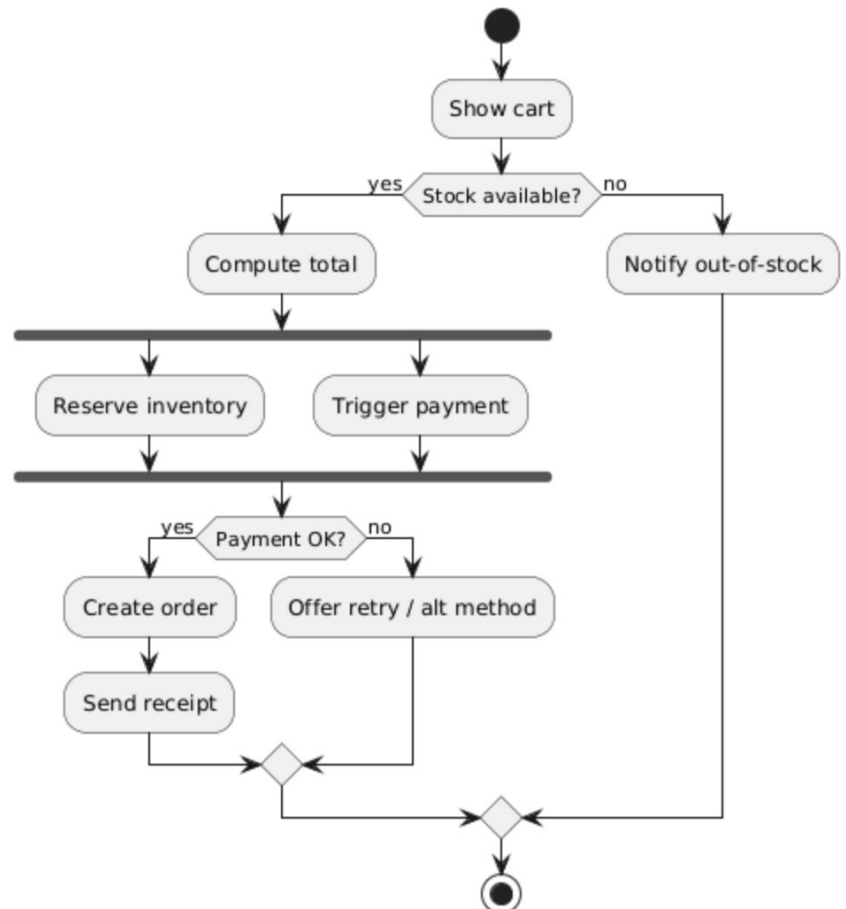


Notes: The rectangle is the system boundary; actors stay outside. Business Owner → Rental Clerk is an actor generalization (the owner can do every clerk task plus UC-9 and UC-12). «include» points from the base to the always-required use case (UC-1 and UC-11 always run UC-4). «extend» points from the optional use case to the base, with its condition (UC-5 extends UC-1 when the customer pays at pickup, and UC-2 when a balance is outstanding). UC-5 is started by the Rental Clerk inside UC-1 or UC-2, so it has no separate actor line. Won't-have use cases UC-13 and UC-14 are not part of this release, so they are not drawn.

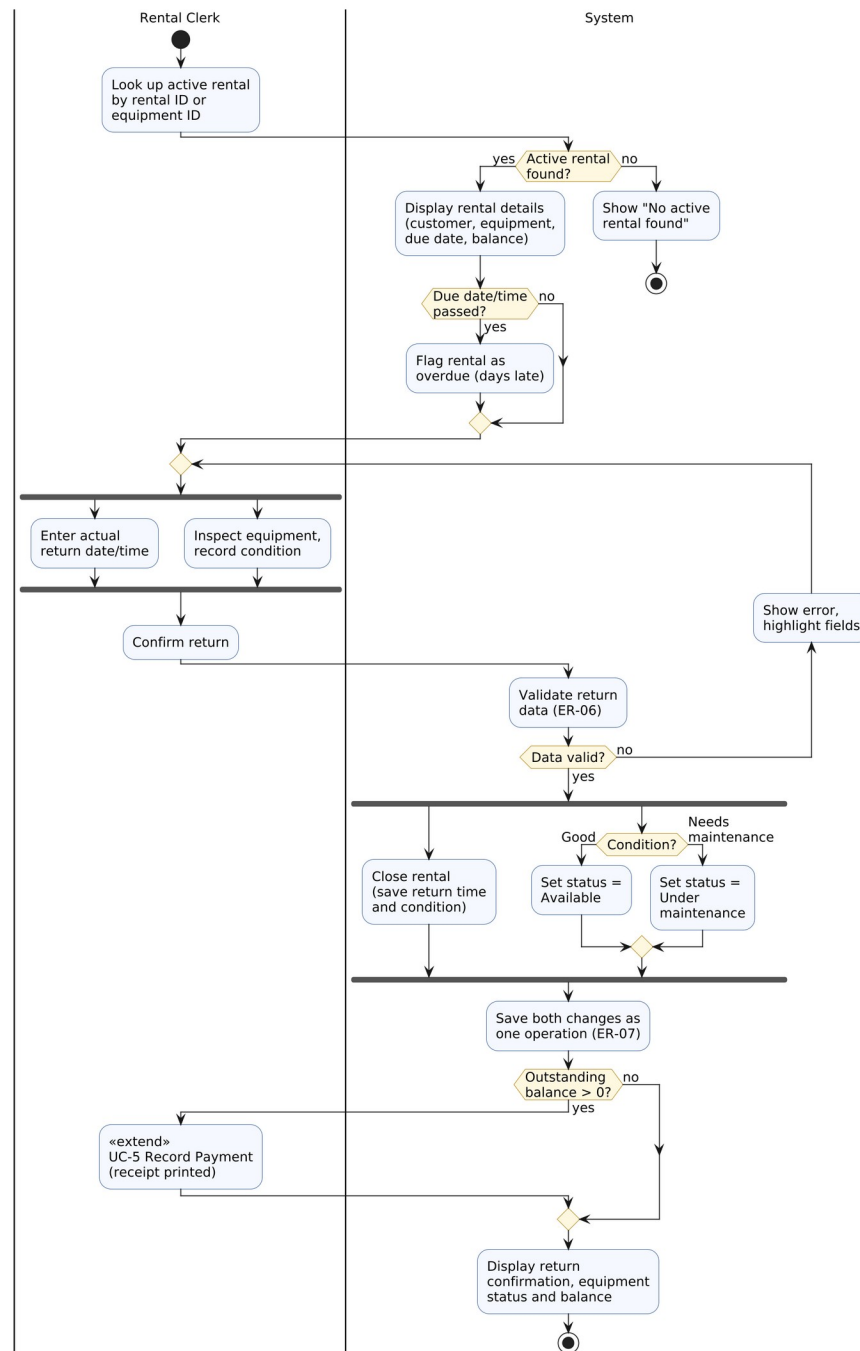
Step 5 — Elaborate Behavior

5A. Activity Diagram for the Complex Case (25 min) Model the **workflow** including decisions, parallelism (fork/join), and guards.

```
@startuml
start
:Show cart;
if (Stock available?) then (yes)
:Compute total;
fork
:Reserve inventory;
fork again
:Trigger payment;
end fork
if (Payment OK?) then (yes)
:Create order;
:Send receipt;
else (no)
:Offer retry / alt method;
endif
else (no)
:Notify out-of-stock;
endif
stop
@enduml
```

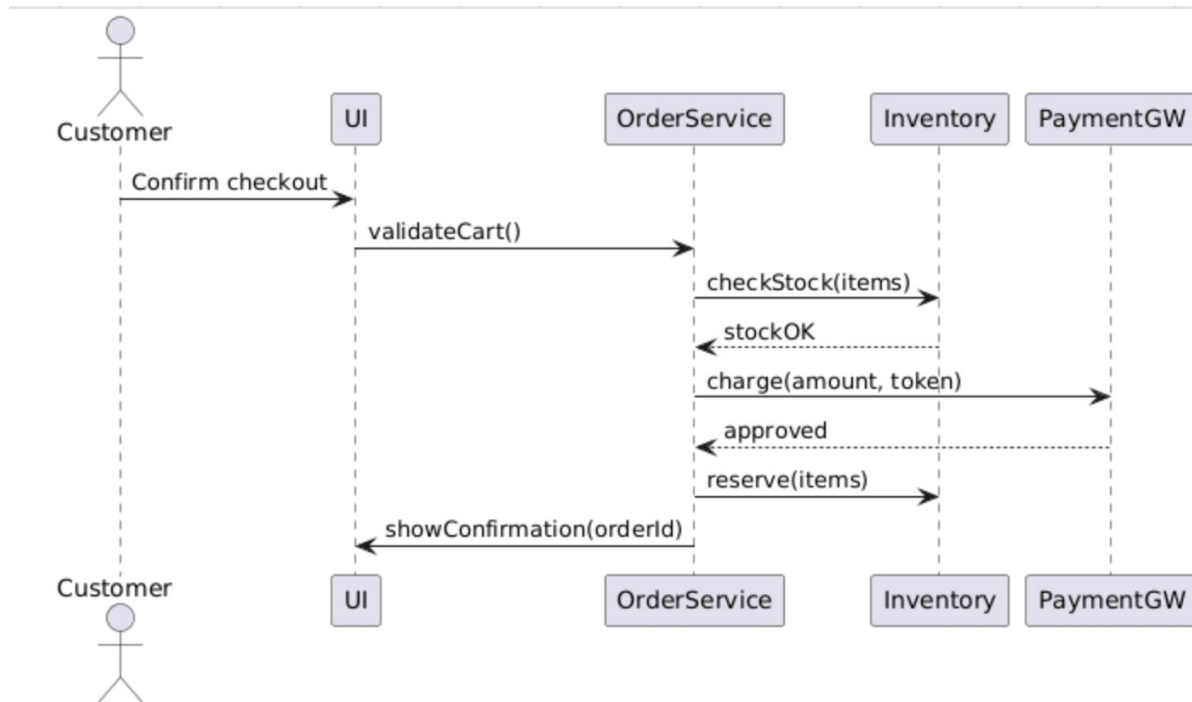


D4 — Activity Diagram: UC-2 Return Equipment (our most complex use case)



Notes: Swimlanes show who does each action. Decisions use guards that match the UC-2 extensions: [Active rental found?] = 1a, [Due date/time passed?] = 2a, [Data valid?] = 6a, [Condition?] = 4a, [Outstanding balance > 0?] = 8a. Fork/join: the clerk can enter the return time and inspect the item in any order, and the system closes the rental and updates the equipment status together (one operation, ER-07).

5B. Sequence Diagram for Main Success (25 min) Capture object interactions and lifelines.



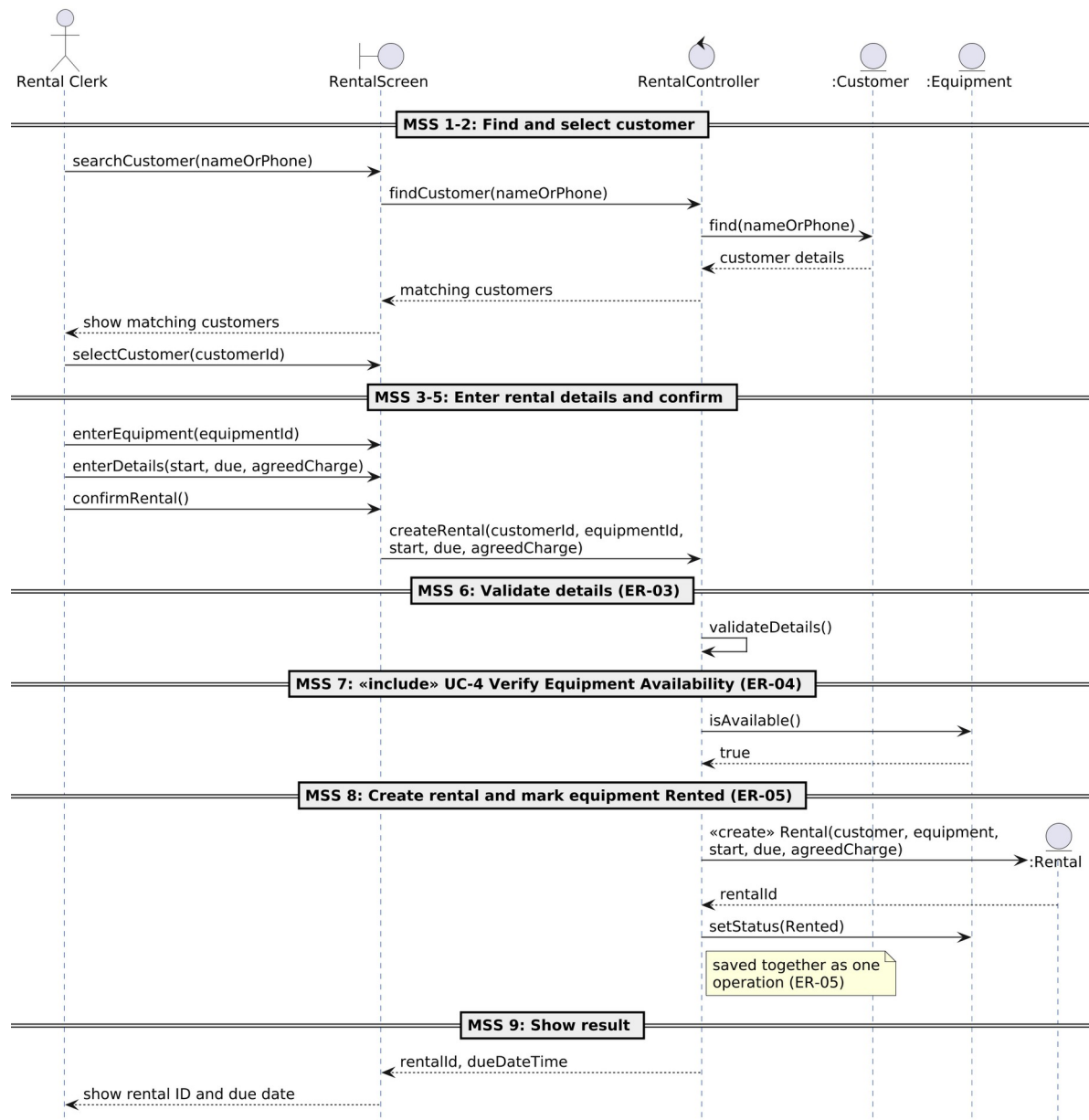
```

@startuml
actor Customer
participant UI
participant OrderService
participant Inventory
participant PaymentGW as PG
  
```

```

Customer -> UI: Confirm checkout
UI -> OrderService: validateCart()
OrderService -> Inventory: checkStock(items)
Inventory --> OrderService: stockOK
OrderService -> PG: charge(amount, token)
PG --> OrderService: approved
OrderService -> Inventory: reserve(items)
OrderService -> UI: showConfirmation(orderId)
@enduml
  
```

D5 — Sequence Diagram: UC-1 Rent Equipment, main success scenario



Notes: Each divider names the MSS steps of UC-1 it covers (steps 1–9). Lifelines use boundary/control/entity objects; the entity lifelines (:Customer, :Equipment, :Rental) are classes from the domain model (D6). Solid arrows are calls, dashed arrows are returns.

Step 6 — Domain Model (Class Diagram) (30 min)

Extract **domain concepts** (nouns) and relationships (associations, multiplicities, composition) from your requirements and use cases. Avoid technical/database details.

```

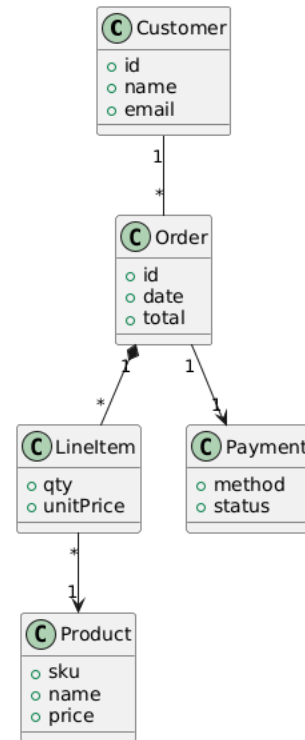
@startuml
class Customer {
    +id
    +name
    +email
}
class Order {
    +id
    +date
    +total
}
class LineItem {
    +qty
    +unitPrice
}
class Product {
    +sku
    +name
    +price
}
class Payment {
    +method
    +status
}

```

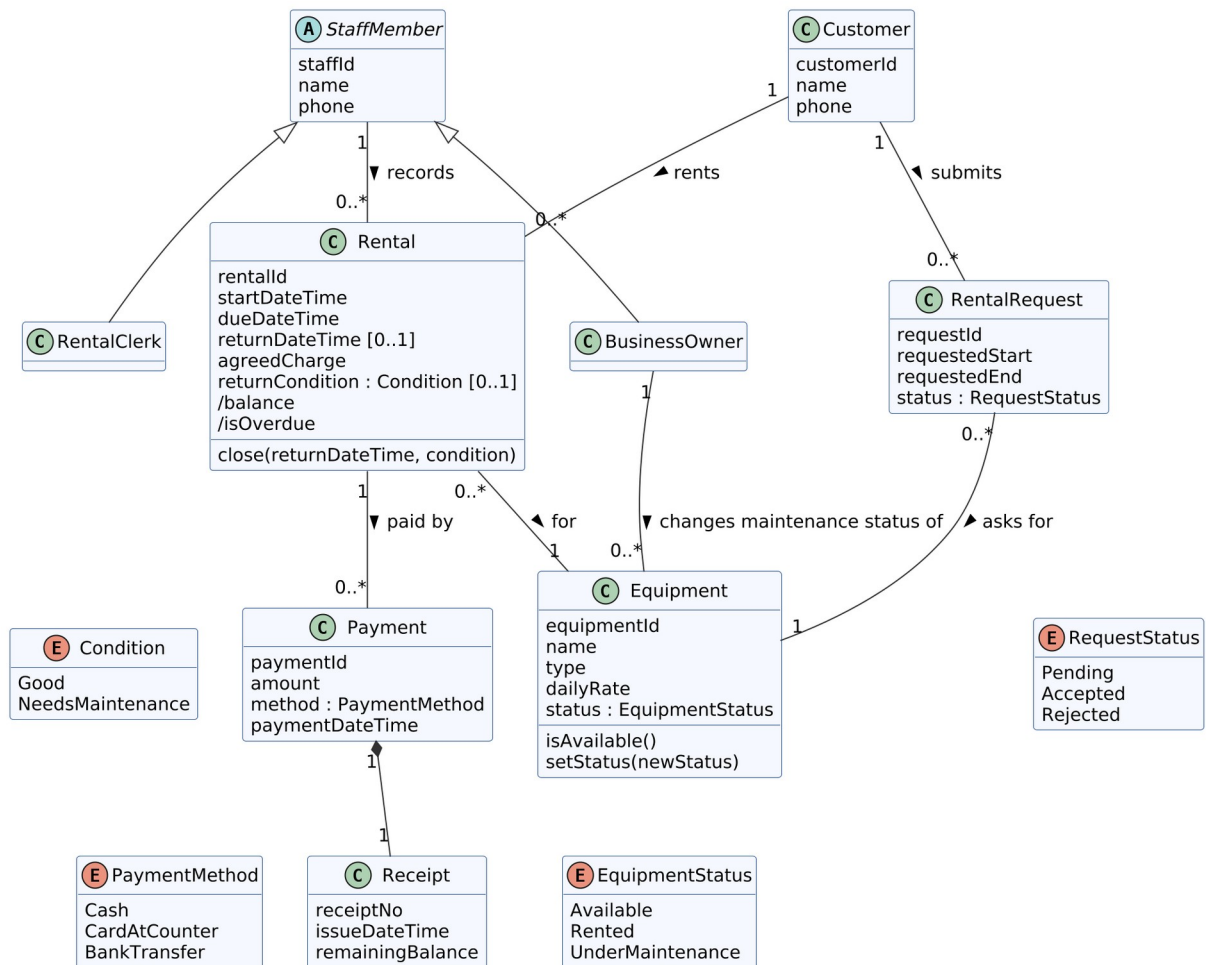
```

Customer "1" -- "*" Order
Order "1" *-- "*" LineItem
LineItem "*" --> "1" Product
Order "1" --> "1" Payment
@enduml
@

```



D6 — EquRental Domain Model (Class Diagram)



Notes: Concepts come from the nouns in our requirements and use cases (no screens or database tables). A Customer rents many Rentals; each Rental is for exactly one Equipment item (one item per rental, ER-03) and is recorded by one StaffMember. A Rental can be paid in several Payments; each Payment owns one Receipt (composition, ER-17). /balance (= agreed charge – payments, ER-16) and /isOverdue (due date passed and not returned, ER-08) are derived. RentalClerk and BusinessOwner specialize StaffMember, matching the actor generalization in D3. RentalRequest supports UC-11.

Step 7 — Consistency Checks (10 min)

- Every **use case** is traceable to at least one **actor** and appears in the **Use Case Diagram**.
- The **Sequence Diagram** messages align with the **Main Success Scenario** steps.
- The **Activity Diagram** branches match **Extensions/guards** in the use case.

- All **domain classes** appear in at least one use case/sequence; names consistent.
- Non-functional constraints in the use case appear as notes/stereotypes where relevant.

D7 — Our consistency checklist (evidence)

Check	Status	Evidence
Every use case traceable to an actor and in the UCD	✓ Done	All 12 M/S/C use cases are in D3. Each links to an actor directly, by inheritance (Business Owner), or through its base use case (UC-4 included by UC-1/UC-11; UC-5 extends UC-1/UC-2). UC-13/UC-14 are Won't and listed only in D1.
Sequence messages align with MSS	✓ Done	D5 dividers follow UC-1 steps 1–9: confirmRental() = step 5, validateDetails() = step 6, isAvailable() = step 7 («include» UC-4), «create» Rental + setStatus(Rented) = step 8, rental ID and due date = step 9.
Activity branches match extensions/guards	✓ Done	D4 guards map to UC-2 extensions 1a, 2a, 4a, 6a, 8a (see notes under D4).
Domain classes used in use cases/sequence; names consistent	✓ Done	Customer, Equipment, Rental are lifelines in D5; Payment and Receipt appear in UC-5; RentalRequest in UC-11; StaffMember/RentalClerk/BusinessOwner match the actors. Status names (Available, Rented, Under maintenance) are identical in D2, D4, D6 and the state machine.
Non-functional constraints shown as notes/stereotypes	✓ Done	“One operation” notes (ER-05, ER-07) in D4 and D5; «include»/«extend» stereotypes in D3; bilingual UI and 3-second response are in the Special Requirements of UC-1 and UC-2.

Step 8 — Peer Review (15 min)

Swap with another team and complete the checklist below.

Team being reviewed: New Group 8 EquRental — Alaa Alqurashi, Faisal Alburti, Hashim Alsayed

Reviewing team: _____ **Reviewer name(s):** _____

Review date: _____

The checklist below is completed by the reviewing team (ticks and comments left blank for them).

Peer-Review Checklist –

- [] Actors cover all personas and external systems –
- [] Use case names are goal-level and unambiguous –
- [] Includes/extends used sparingly and correctly –
- [] Activity/Sequence diagrams match narratives –
- [] Class diagram avoids UI/DB details; multiplicities make sense –

- [] Clear pre/post-conditions and success guarantees –
- [] Consistency across all artifacts –
- [] Clarity, readability, and professional style

Reviewer comments: _____

Changes we made after the review: _____

Grading Rubric (100 pts)

Component	Criteria	Pts
Actors & Use Case List	Complete, prioritized, correct scoping	15
Fully Dressed Use Cases (×2)	Preconditions, MSS, extensions, NFRs	25
Use Case Diagram	Correct notation, relationships, boundary	15
Activity Diagram	Flow accuracy, guards, concurrency	15
Sequence Diagram	Lifelines, messages align with MSS	15
Domain Model	Correct concepts/relations, multiplicities	10
Consistency + Peer Review	Checklist evidence, coherence	5

Submission & Naming

- File: TeamName_UML_Lab.pdf. If using PlantUML, append a ZIP with .puml sources.
-

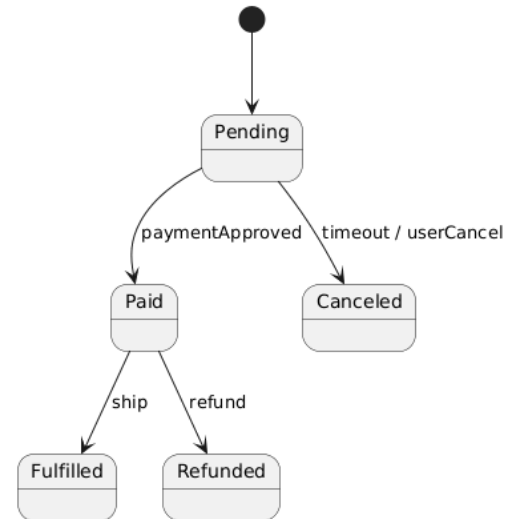
Extension (Optional, for +5 pts each)

- **State Machine Diagram** for a key entity (e.g., *Order* lifecycle)

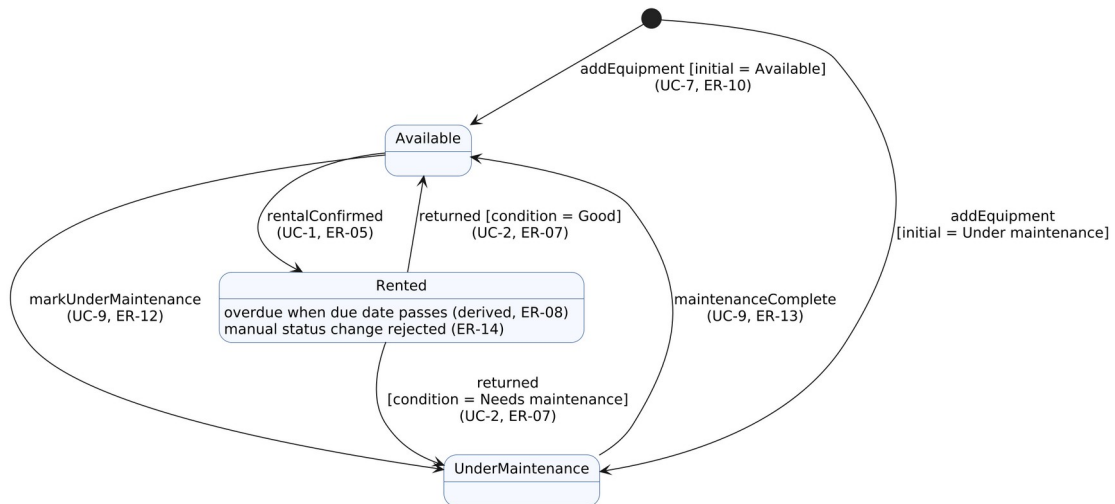
- **System Sequence Diagram** for an alternative flow (e.g., failed payment)

State Machine Example

```
@startuml
[*] --> Pending
Pending --> Paid : paymentApproved
Paid --> Fulfilled : ship
Paid --> Refunded : refund
Pending --> Canceled : timeout / userCancel
@enduml
```

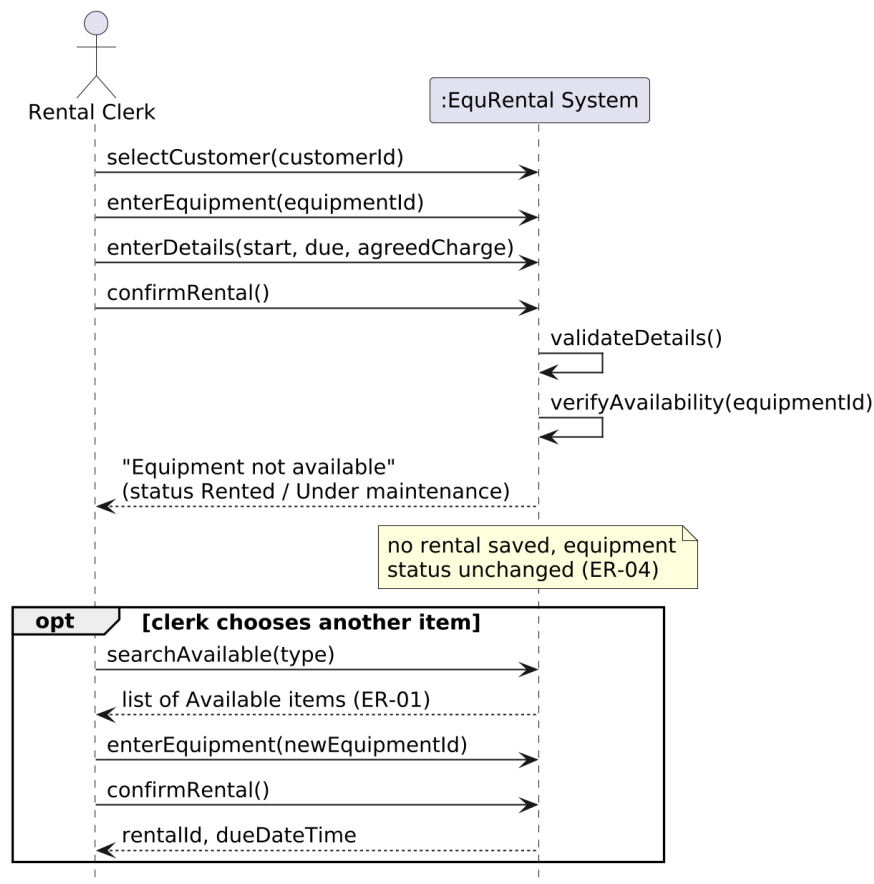


Our Extension 1 — State Machine Diagram: Equipment lifecycle



Notes: Uses the same three statuses as the requirements and domain model.
 "Overdue" is not a separate state; it is derived while the item is Rented (ER-08).

Our Extension 2 — System Sequence Diagram: UC-1 alternative flow 7a (equipment not available)



Quick Reference — When to Use Which UML

- **Use Case Diagram:** scope & goals from user perspective
- **Activity Diagram:** end-to-end workflow with decisions/parallelism
- **Sequence Diagram:** object interactions over time for a scenario
- **Class Diagram:** static domain structure and relationships
- **State Machine:** lifecycle of a complex entity

Tip: Start from personas → goals → use cases; only then elaborate behavior and structure. Keep names consistent across all artifacts.

Appendix — PlantUML Source (our diagrams)

D3 Use Case Diagram (d3_usecase.puml)

```
@startuml d3_usecase
left to right direction

actor "Rental Clerk" as Clerk
actor "Business Owner" as Owner
```

```

actor "Customer" as Cust

rectangle "Equipment Rental Management System (EquRental)" {
    usecase "UC-1 Rent Equipment" as UC1
    usecase "UC-2 Return Equipment" as UC2
    usecase "UC-3 Search Available\nequipment" as UC3
    usecase "UC-4 Verify Equipment\nAvailability" as UC4
    usecase "UC-5 Record Payment" as UC5
    usecase "UC-6 Manage Customer\nRecords" as UC6
    usecase "UC-7 Manage Equipment\nRecords" as UC7
    usecase "UC-8 View Overdue\nRentals" as UC8
    usecase "UC-9 Update Maintenance\nStatus" as UC9
    usecase "UC-10 Browse Equipment\nCatalog" as UC10
    usecase "UC-11 Submit Rental\nRequest" as UC11
    usecase "UC-12 View Business\nReports" as UC12
}

Clerk <|-- Owner

Clerk -- UC3
Clerk -- UC6
Clerk -- UC1
Clerk -- UC2
Clerk -- UC7
Clerk -- UC8
Owner -- UC9
Owner -- UC12

UC1 ..> UC4 : <<include>>
UC1 <.. UC5 : <<extend>>\n[customer pays\nat pickup]
UC2 <.. UC5 : <<extend>>\n[balance\noutstanding]
UC12 -[hidden]- UC11
UC6 -[hidden]- UC10
UC11 .[norank].> UC4 : <<include>>
UC11 -- Cust
UC10 -- Cust
@enduml

```

D4 Activity Diagram (d4_activity_return.puml)

```

@startuml d4_activity_return

|Rental Clerk|
start
:Look up active rental\nby rental ID or\nequipment ID;
|System|
if (Active rental\nfound?) then (yes)
    :Display rental details\n(customer, equipment,\ndue date, balance);
    if (Due date/time\npassed?) then (yes)
        :Flag rental as\noverdue (days late);
    else (no)
    endif
else (no)
    :Show "No active\nrental found";
    stop
endif
|Rental Clerk|
repeat
    fork
        :Enter actual\nreturn date/time;
    fork again
        :Inspect equipment,\nrecord condition;
    end fork
    :Confirm return;
|System|
    :Validate return
    data (ER-06);
backward:Show error,\nhighlight fields;
repeat while (Data valid?) is (no) not (yes)
    fork
        :Close rental\n(save return time\nand condition);
    fork again
        if (Condition?) then (Good)
            :Set status =\nAvailable;
        else (Needs\nmaintenance)
            :Set status =\nUnder\nmaintenance;
        endif
    end fork
    :Save both changes as
    one operation (ER-07);
    if (Outstanding\nbalance > 0?) then (yes)

```

```

    |Rental Clerk|
    :<<extend>>\nUC-5 Record Payment\n(receipt printed);
else (no)
endif
|System|
:Display return\nconfirmation, equipment\nstatus and balance;
stop
@enduml

```

D5 Sequence Diagram (d5_sequence_rent.puml)

```

@startuml d5_sequence_rent
hide footbox

actor "Rental Clerk" as Clerk
boundary "RentalScreen" as UI
control "RentalController" as Ctrl
entity ":Customer" as Cust
entity ":Equipment" as Eq
entity ":Rental" as Rent

== MSS 1-2: Find and select customer ==
Clerk -> UI : searchCustomer(nameOrPhone)
UI -> Ctrl : findCustomer(nameOrPhone)
Ctrl -> Cust : find(nameOrPhone)
Cust --> Ctrl : customer details
Ctrl --> UI : matching customers
UI --> Clerk : show matching customers
Clerk -> UI : selectCustomer(customerId)

== MSS 3-5: Enter rental details and confirm ==
Clerk -> UI : enterEquipment(equipmentId)
Clerk -> UI : enterDetails(start, due, agreedCharge)
Clerk -> UI : confirmRental()
UI -> Ctrl : createRental(customerId, equipmentId,\nstart, due, agreedCharge)

== MSS 6: Validate details (ER-03) ==
Ctrl -> Ctrl : validateDetails()

== MSS 7: <<include>> UC-4 Verify Equipment Availability (ER-04) ==
Ctrl -> Eq : isAvailable()
Eq --> Ctrl : true

== MSS 8: Create rental and mark equipment Rented (ER-05) ==
Ctrl -> Rent ** : <<create>> Rental(customer, equipment,\nstart, due, agreedCharge)
Rent --> Ctrl : rentalId
Ctrl -> Eq : setStatus(Rented)
note right of Ctrl : saved together as one\noperation (ER-05)

== MSS 9: Show result ==
Ctrl --> UI : rentalId, dueDateTime
UI --> Clerk : show rental ID and due date
@enduml

```

D6 Domain Model (d6_domain_model.puml)

```

@startuml d6_domain_model
hide empty methods

abstract class StaffMember {
    staffId
    name
    phone
}
class RentalClerk
class BusinessOwner

class Customer {
    customerId
    name
    phone
}

class Equipment {
    equipmentId
    name
    type
    dailyRate
    status : EquipmentStatus
}

```

```

    isAvailable()
    setStatus(newStatus)
}

class Rental {
    rentalId
    startDateTime
    dueDateTime
    returnDateTime [0..1]
    agreedCharge
    returnCondition : Condition [0..1]
    /balance
    /isOverdue
    close(returnDateTime, condition)
}

class Payment {
    paymentId
    amount
    method : PaymentMethod
    paymentDateTime
}

class Receipt {
    receiptNo
    issueDateTime
    remainingBalance
}

class RentalRequest {
    requestId
    requestedStart
    requestedEnd
    status : RequestStatus
}

enum EquipmentStatus {
    Available
    Rented
    UnderMaintenance
}

enum Condition {
    Good
    NeedsMaintenance
}

enum PaymentMethod {
    Cash
    CardAtCounter
    BankTransfer
}

enum RequestStatus {
    Pending
    Accepted
    Rejected
}

StaffMember <|-- RentalClerk
StaffMember <|-- BusinessOwner

Customer "1" -- "0..*" Rental : rents >
Rental "0..*" -- "1" Equipment : for >
StaffMember "1" -- "0..*" Rental : records >
Rental "1" -- "0..*" Payment : paid by >
Payment "1" *-- "1" Receipt
Customer "1" -- "0..*" RentalRequest : submits >
RentalRequest "0..*" -- "1" Equipment : asks for >
BusinessOwner "1" -- "0..*" Equipment : changes maintenance status of >

Equipment -[hidden]- EquipmentStatus
Rental -[hidden]- Condition
Payment -[hidden]- PaymentMethod
RentalRequest -[hidden]- RequestStatus
@enduml

```

Extension: State Machine (ext_state_equipment.puml)

```

@startuml ext_state_equipment
[*] --> Available : addEquipment [initial = Available]\n(UC-7, ER-10)
[*] --> UnderMaintenance : addEquipment\n[initial = Under maintenance]
Available --> Rented : rentalConfirmed\n(UC-1, ER-05)
Rented --> Available : returned [condition = Good]\n(UC-2, ER-07)

```

Rented --> UnderMaintenance : returned\n[condition = Needs maintenance]\n(UC-2, ER-07)
Available --> UnderMaintenance : markUnderMaintenance\n(UC-9, ER-12)
UnderMaintenance --> Available : maintenanceComplete\n(UC-9, ER-13)
Rented : overdue when due date passes (derived, ER-08)
Rented : manual status change rejected (ER-14)
@enduml

Extension: System Sequence Diagram (ext_ssd_unavailable.puml)

```
@startuml ext_ssd_unavailable
hide footbox
actor "Rental Clerk" as Clerk
participant "EquRental System" as Sys

Clerk -> Sys : selectCustomer(customerId)
Clerk -> Sys : enterEquipment(equipmentId)
Clerk -> Sys : enterDetails(start, due, agreedCharge)
Clerk -> Sys : confirmRental()
Sys -> Sys : validateDetails()
Sys -> Sys : verifyAvailability(equipmentId)
Sys --> Clerk : "Equipment not available"\n(status Rented / Under maintenance)
note over Sys : no rental saved, equipment\nstatus unchanged (ER-04)
opt clerk chooses another item
    Clerk -> Sys : searchAvailable(type)
    Sys --> Clerk : list of Available items (ER-01)
    Clerk -> Sys : enterEquipment(newEquipmentId)
    Clerk -> Sys : confirmRental()
    Sys --> Clerk : rentalId, dueDateTime
end
@enduml
```